

INSTITUTO FEDERAL

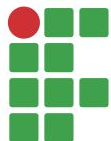
Mato Grosso do Sul

Construção de Páginas Web III

Milena Alegre

Checklist de Conteúdos

- 1. Manipulação de Arquivos
- ❑ 2. Sessões e Cookies
- ❑ 3. Geração de Relatórios
- ❑ 4. Criação de Aplicação Web Completa (S.O.L.I.D)
- ❑ 5. Filtro de Acesso



INSTITUTO FEDERAL

Mato Grosso do Sul

Manipulação de Arquivos

Síncrono vs Assíncrono, Persistência com JSON, Módulo Path, Tratamento de Erros e Upload de Imagens na Prática.

Recapitulando

Passo 1 & 2: Gravação

Formulário HTML envia dados via **POST /estudo** para o servidor Express.

Uso do `fs.appendFileSync` para salvar cada anotação em uma nova linha no arquivo `diario.txt`.

[MATÉRIA]: Resumo do aprendizado...

Passo 3 & Bônus: Leitura

Rota **GET /historico** realiza a leitura do `diario.txt` usando `fs.readFileSync`.

Os dados salvos são formatados e exibidos diretamente na tela do navegador com estilo CSS customizado.

utf-8 -> HTML estilizado

Modulo fs



Manipulação Direta

Permite criar, ler, atualizar, renomear e deletar arquivos diretamente no sistema operacional do servidor.



Duas Abordagens

Oferece métodos ****Síncronos**** (bloqueantes) e ****Assíncronos**** (baseados em Callbacks ou Promises/async-await).



Base para Bancos

Compreender o `fs` é essencial para entender como bancos de dados (como SQLite ou MongoDB) gravam dados em disco.

Sync x Async

Síncrono (Sync)

BLOQUEANTE

Executa linha por linha. A aplicação ****PARA**** e aguarda o término do I/O (leitura/escrita no disco) para avançar.

Uso ideal: Scripts de inicialização do servidor, leitura de configurações na subida da aplicação.

Assíncrono (Async)

NÃO-BLOQUEANTE

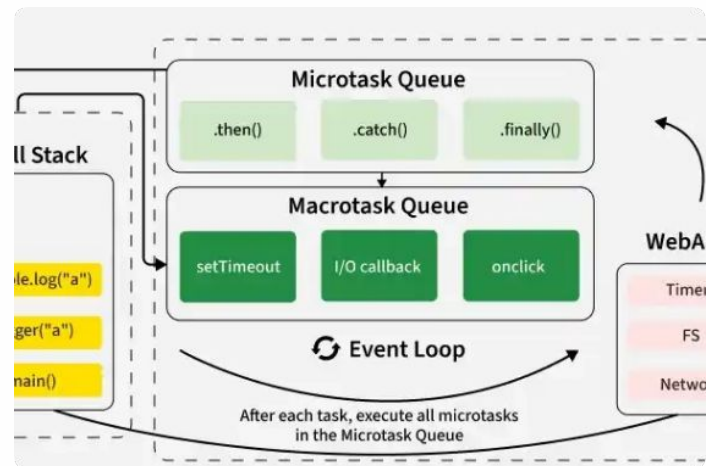
Solicita a operação ao SO e ****LIBERA**** o Node.js imediatamente para processar outras requisições.

Uso ideal: Rotas de APIs, manipulação de arquivos por usuários, servidores web com múltiplos acessos.

Sync x Async – Analogia

 **Caixa Eletrônico (Síncrono):** Atende uma pessoa por vez. A fila inteira fica travada esperando o cliente concluir a operação antes da próxima pessoa se aproximar.

 **Garçom do Restaurante (Assíncrono):** Anota seu pedido, envia para a cozinha e atende outras mesas enquanto o prato é preparado. Quando a comida fica pronta, ele a entrega.



O que acontece quando você usa `readFileSync` em produção?

O Node.js utiliza o **Event Loop** rodando em uma única thread para atender requisições.

Se o usuário **A** solicitar a leitura de um arquivo de 50MB via código Síncrono, os usuários **B**, **C** e **D** ficarão congelados aguardando.

Resultado: Queda severa de performance, timeout na aplicação e péssima experiência do usuário.

O que acontece quando você usa `readFileSync` em produção?

O Node.js utiliza o **Event Loop** rodando em uma única thread para atender requisições.

Se o usuário **A** solicitar a leitura de um arquivo de 50MB via código Síncrono, os usuários **B**, **C** e **D** ficarão congelados aguardando.

Resultado: Queda severa de performance, timeout na aplicação e péssima experiência do usuário.

Uma **Promise** (Promessa) no JavaScript é um objeto que representa a entrega futura de um resultado — seja o sucesso de uma operação ou o motivo de sua falha. Ela é o pilar da programação assíncrona moderna, permitindo executar tarefas demoradas (como ler arquivos ou consultar um banco de dados) sem travar a aplicação.

```
const fs = require('fs');

console.log('1. Inicio');
const dados = fs.readFileSync('diario.txt', 'utf8');
console.log('2. Conteudo lido');
console.log('3. Fim');
```

```
// Saida SEMPRE garantida:
// 1. Inicio → 2. Conteudo lido → 3. Fim
```

1. Teste Síncrono (Bloqueia)

```
const fs = require('fs').promises;

console.log('1. Inicio');
fs.readFile('diario.txt', 'utf8')
  .then(dado ⇒ console.log('2. Conteudo lido'));
console.log('3. Fim');
```

```
// Saida no Console:
// 1. Inicio → 3. Fim → 2. Conteudo lido!
```

2. Teste Assíncrono (Não Bloqueia)

Assincronismo

1. Callbacks

Passagem de funções de retorno.
Pode gerar o famoso "Callback Hell" e dificultar a leitura do código.

2. Promises

Introdução dos métodos `.then()` e `.catch()` para encadeamento mais limpo e organizado.

3. Async / Await

A forma moderna (ES6+). Permite escrever código assíncrono com sintaxe limpa parecida com a síncrona.

4. fs.promises

Módulo nativo do Node.js que disponibiliza métodos do File System retornando Promises nativas.

Lendo arquivos

Importamos o submódulo com `const fs = require('fs').promises;`

Utilizamos `await fs.readFile(caminho, 'utf-8')` dentro de uma função marcada como `async`.

```
const fs = require('fs').promises;

async function lerDiario() {
  try {
    const conteudo = await fs.readFile('diario.txt', 'utf-8');
    console.log('Conteúdo do diário:', conteudo);
  } catch (erro) {
    console.error('Falha ao ler o arquivo:', erro.message);
  }
}

lerDiario();
```

Escrita Assincrona

fs.writeFile()

Substitui / Sobrescreve todo o conteúdo do arquivo existente.

Se o arquivo não existir, ele cria um novo.

```
await fs.writeFile(  
  'diario.txt',  
  'Novo texto'  
);
```

fs.appendFile()

Anexa / Adiciona o novo conteúdo no final do arquivo sem apagar o que já existe lá dentro.

```
await fs.appendFile(  
  'diario.txt',  
  '\nNova linha'  
);
```

Sobre o Path



Separadores do SO

Windows usa barra invertida (\) enquanto Linux/macOS usam barra normal (/). O `path` resolve isso automaticamente.



Variável `\_\_dirname`

Aponta para o caminho absoluto da pasta onde o arquivo em execução atual se encontra armazenado.



Sintaxe `path.join()`

```
const path = require('path');  
const rotaArquivo = path.join(  
  __dirname,  
  'dados',  
  'diario.txt'  
);
```

Erros Comuns

Código do Erro	Significado do Código	Causa Provável	Ação Recomendada
ENOENT	Error NO ENTry	Arquivo ou diretório especificado não existe no caminho informado.	Criar o arquivo ou verificar se o caminho no <code>`path.join`</code> está correto.
EACCES	Error ACCESS denied	Servidor sem permissão para ler ou gravar na pasta.	Ajustar permissões do sistema operacional da pasta/arquivo.
EEXIST	Error EXISts	Tentativa de criar uma pasta/arquivo que já existe.	Verificar se a pasta já existe antes de tentar rodar <code>`mkdir`</code> .

Ciclo de Vida JSON



1. Leitura e Parse

Lê a string do arquivo com `fs.readFile()` e converte para Array/Objeto JS com `JSON.parse()`.



2. Manipulação

Modifica os dados em memória usando métodos do JS como `.push()`, `.filter()` ou `.find()`.



3. Stringify e Escrita

Converte o Objeto de volta para Texto com `JSON.stringify()` e grava no arquivo com `fs.writeFile()`.

Ciclo de Vida JSON

- O arquivo em disco guarda apenas **Texto Puro (String)**.
- Para conseguir fazer buscas e adicionar itens, precisamos converter esse texto em um Objeto / Array de verdade.

```
// 1. Le o texto puro do arquivo JSON
const textoPuro = await fs.readFile('estudos.json', 'utf-8');

// 2. Transforma a String em um Array/Objeto de verdade no JavaScript
const listaDeEstudos = JSON.parse(textoPuro);

// Agora podemos usar metodos do JS!
console.log(listaDeEstudos.length);
```

JSON.parse()

Manipulando dado na memória

Adicionar (.push)

Insere o novo registro recebido do formulário.

```
lista.push({  
  materia: 'Node.js',  
  resumo: 'JSON'  
});
```

Filtrar (.filter)

Remove itens mantendo os demais.

```
const filtrados = lista.filter(  
  item => item.id !== id  
);
```

Buscar (.find)

Localiza um item específico por campo.

```
const item = lista.find(  
  i => i.materia === 'Math'  
);
```

Ciclo de Vida JSON

→ Para gravar no arquivo novamente, precisamos converter o Objeto JavaScript em String.

★ **Dica de Ouro:** Use os parâmetros adicionais do `JSON.stringify(dados, null, 2)` para indentar e salvar o JSON bonitinho e legível!

```
const novadata = { materia: 'Express', aprendizado: 'Uploads' };  
listaDeEstudos.push(novadata);
```

```
// Converte para String FORMATADA com recuo de 2 espacos
```

```
const jsonFormatado = JSON.stringify(listaDeEstudos, null, 2);
```

```
// Sobrescreve o arquivo com a lista atualizada
```

```
await fs.writeFile('estudos.json', jsonFormatado);
```

JSON.stringify()

Salvando Dados Pelo Front

Quando o usuário envia um formulário com `req.body`, capturamos os dados no Express e integramos ao fluxo do arquivo JSON:

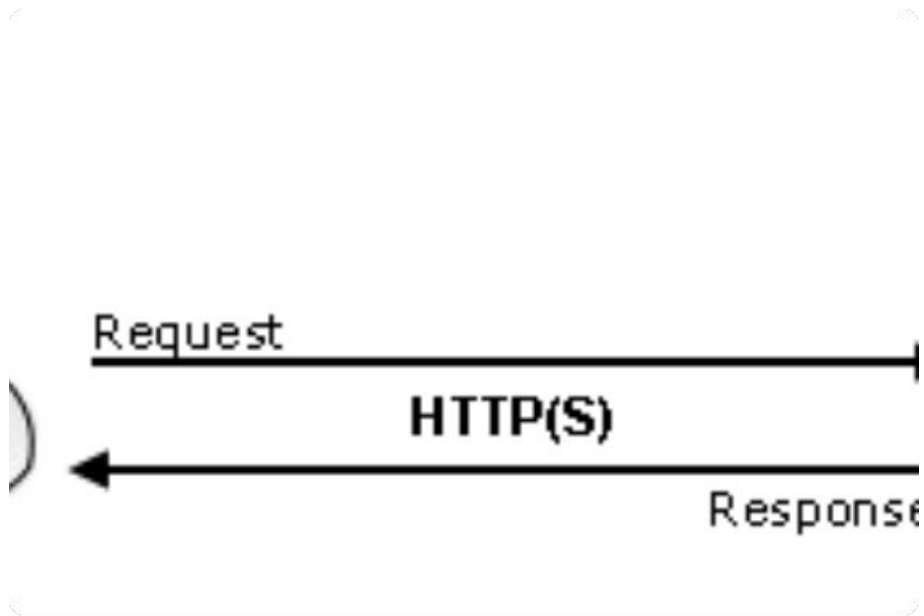
Recebe `req.body` na rota POST.

Lê o arquivo `banco.json` existente.

Executa o `JSON.parse()`.

Dá um `.push(req.body)`.

Salva com `fs.writeFile()`.



Salvando imagens

Como receber fotos, ícones de avatar e documentos enviados pelo formulário e armazená-los no servidor.

Por que **req.body** Não Recebe Imagens?

Campos de Texto Comuns

Enviados via cabeçalho

application/x-www-form-urlencoded ou

application/json.

O Express consegue decodificar como texto simples e popular o objeto req.body nativamente.

Arquivos de Mídia e Imagens

*Imagens são **dados binários** complexos. Precisam ser enviadas via formulário em partes: multipart/form-data.*

*O Express **NÃO** processa isso sozinho. Precisamos do middleware **Multer**!*

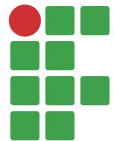
Salvando imagens

Atributo enctype do Formulário	Tipo de Dado Suportado	Middleware no Express
application/x-www-form-urlencoded	Apenas texto simples de inputs de formulário padrão.	<code>express.urlencoded({ extended: true })</code>
application/json	Objetos e Arrays JSON enviados via API / Fetch.	<code>express.json()</code>
multipart/form-data	Textos E Arquivos Binários (Fotos, PDFs, Avatares).	Multer (lib externa)

Checklist de Boas Práticas

-  **Sempre use Async em Rotas:** Prefira `fs.promises` com `async/await` para não travar a aplicação.
-  **Módulo Path Sempre:** Sempre construa caminhos com `path.join(__dirname, ...)`.
-  **Trate Exceções:** Envolver leituras e escritas em blocos `try/catch` e verifique erros como `ENOENT`.
-  **Salve Apenas Caminhos de Imagem:** Nunca guarde a imagem binária inteira dentro do arquivo JSON. Salve a imagem na pasta `/uploads` e grave apenas o caminho `/uploads/nome.jpg` no JSON.

milena.alegre@ifms.edu.br



INSTITUTO FEDERAL
Mato Grosso do Sul